# Introduction

## Domain-Specific Area

This project focuses on the consumer electronics domain, specifically laptop pricing prediction in the retail market. The global laptop market is valued at over $100 billion annually, with consumers and businesses making purchasing decisions based on technical specifications and pricing. Understanding the factors that influence laptop prices is crucial for manufacturers, retailers, and consumers.

Linear regression is suitable for this domain because laptop prices show an approximately linear relationship with key specifications like RAM, processor speed, storage capacity, and screen size. Higher specifications generally command higher prices in a proportional manner, making linear regression an appropriate and interpretable modeling approach.

Objectives: The primary objectives of this project are:

1. To develop a predictive model that accurately estimates laptop prices based on technical specifications.
2. to identify which specifications have the strongest influence on pricing
3. To provide insights for consumers making informed purchasing decisions
4. To help retailers with competitive pricing strategies

This project utilizes the "Uncleaned Laptop Price Dataset" from Kaggle, which provides real-world laptop specifications and pricing data in its raw, unprocessed form. The dataset's intentionally messy format makes it ideal for demonstrating comprehensive preprocessing, a critical component of any real-world data. The dataset includes numerous records and a mix of numerical, categorical, and textual features such as RAM size, CPU description, storage configuration, screen resolution, brand, and weight.

The dataset is not in 1NF as several attributes are embedded within text fields, eg(8GB RAM + 256GB SSD). This makes it ideal for demonstrating data cleaning, feature extraction, and preprocessing, which are critical steps. This dataset works well with linear regression because:

- The target variable(price) is contnoius
- Many features exhibit linear or near-linear relationships with price
- The dataset size is manageable, which makes it efficient

In [1]:
```
#import required libraries
import pandas as pd
import numpy as np
```

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

df = pd.read_csv('laptopData.csv')
```

1. Loading raw CSV into a pandas dataframe
2. Identifying the missing values and removing rows with a missing target variable (price)
3. Extracting numerical features from text, such as:

- RAM = RAM_GB
- Weight=Weight_kg
- CPU Speed = CPU_GHz
- Stroage = SSD_GB, HDD_GB
- Screen Resolution = Resolution_X, Resoltuion_X

4. Handling missing values with median imputation
5. Converting categorical variables with one-hot encoding

Linear regression requires numerical, independent features. Raw text fields would otherwise prevent model training or introduce bias. Median imputation was chosen because it is robust to outliers in skewed distributions.

In [2]:
```
#visualisation style
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")
```

Applies a consistent visual style and color palette. It shows clear, readable plots imporves interpretability and presentation quality.

In [3]:
```
#displays the number of rows and columns in the dataset
print("Df Shape:", df.shape)
```

```
Df Shape: (1303, 12)
```

In [4]:
```
#to understad the structure
df.head()
```

Out[4]:

| | Unnamed: 0 | Company | TypeName | Inches | ScreenResolution | Cpu | Ram | Memory | |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 0.0 | Apple | Ultrabook | 13.3 | IPS Panel Retina Display 2560x1600 | Intel Core i5 2.3GHz | 8GB | 128GB SSD | In Gr |
| **1** | 1.0 | Apple | Ultrabook | 13.3 | 1440x900 | Intel Core i5 1.8GHz | 8GB | 128GB Flash Storage | In Gr |
| **2** | 2.0 | HP | Notebook | 15.6 | Full HD 1920x1080 | Intel Core i5 7200U 2.5GHz | 8GB | 256GB SSD | In Gr |
| **3** | 3.0 | Apple | Ultrabook | 15.4 | IPS Panel Retina Display 2880x1800 | Intel Core i7 2.7GHz | 16GB | 512GB SSD | R P |
| **4** | 4.0 | Apple | Ultrabook | 13.3 | IPS Panel Retina Display 2560x1600 | Intel Core i5 3.1GHz | 8GB | 256GB SSD | In Gr |

In [5]:

```
df.tail()
```

Out[5]:

| | Unnamed: 0 | Company | TypeName | Inches | ScreenResolution | Cpu | Ram | Memory |
|---|---|---|---|---|---|---|---|---|
| **1298** | 1298.0 | Lenovo | 2 in 1 Convertible | 14 | IPS Panel Full HD / Touchscreen 1920x1080 | Intel Core i7 6500U 2.5GHz | 4GB | 128GB SSD |
| **1299** | 1299.0 | Lenovo | 2 in 1 Convertible | 13.3 | IPS Panel Quad HD+ / Touchscreen 3200x1800 | Intel Core i7 6500U 2.5GHz | 16GB | 512GB SSD |
| **1300** | 1300.0 | Lenovo | Notebook | 14 | 1366x768 | Intel Celeron Dual Core N3050 1.6GHz | 2GB | 64GB Flash Storage |
| **1301** | 1301.0 | HP | Notebook | 15.6 | 1366x768 | Intel Core i7 6500U 2.5GHz | 6GB | 1TB HDD |
| **1302** | 1302.0 | Asus | Notebook | 15.6 | 1366x768 | Intel Celeron Dual Core N3050 1.6GHz | 4GB | 500GB HDD |

In [6]:

```
#check data types and structure
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1303 entries, 0 to 1302
Data columns (total 12 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Unnamed: 0        1273 non-null   float64
 1   Company           1273 non-null   object
 2   TypeName          1273 non-null   object
 3   Inches            1273 non-null   object
 4   ScreenResolution  1273 non-null   object
 5   Cpu               1273 non-null   object
 6   Ram               1273 non-null   object
 7   Memory            1273 non-null   object
 8   Gpu               1273 non-null   object
 9   OpSys             1273 non-null   object
 10  Weight            1273 non-null   object
 11  Price             1273 non-null   float64
dtypes: float64(2), object(10)
memory usage: 122.3+ KB
```

In [8]:
```
#checking for missing values
df.isnull().sum()
```

Out[8]:
```
Unnamed: 0          30
Company             30
TypeName            30
Inches              30
ScreenResolution    30
Cpu                 30
Ram                 30
Memory              30
Gpu                 30
OpSys               30
Weight              30
Price               30
dtype: int64
```

There are 30 rows with missing data across all features. Since these rows are missing the target variable (price), they cannot be used for training or testing. These rows will removed.

In [7]:
```
#removing rows with missing price
df = df.dropna(subset=['Price'])
print("shape after removing incomplete rows:", df.shape)
```

```
shape after removing incomplete rows: (1273, 12)
```

In [8]:
```
#computes summary statistics(mean, std, quartiles) for numerical columns
df.describe()
```

Out[8]:

| | Unnamed: 0 | Price |
|---|---|---|
| **count** | 1273.000000 | 1273.000000 |
| **mean** | 652.674784 | 59955.814073 |
| **std** | 376.493027 | 37332.251005 |
| **min** | 0.000000 | 9270.720000 |
| **25%** | 327.000000 | 31914.720000 |
| **50%** | 652.000000 | 52161.120000 |
| **75%** | 980.000000 | 79333.387200 |
| **max** | 1302.000000 | 324954.720000 |

In [9]:
```
#extracting RAM in GB
#converts RAM from text into int numeric value
df['Ram_GB'] = df['Ram'].str.replace('GB','', regex=False).astype(int)
df[['Ram', 'Ram_GB']].head(10)
```

Out[9]:

| | Ram | Ram_GB |
|---|---|---|
| **0** | 8GB | 8 |
| **1** | 8GB | 8 |
| **2** | 8GB | 8 |
| **3** | 16GB | 16 |
| **4** | 8GB | 8 |
| **5** | 4GB | 4 |
| **6** | 16GB | 16 |
| **7** | 8GB | 8 |
| **8** | 16GB | 16 |
| **9** | 8GB | 8 |

In [10]:

```
#extracting Weight in kg
#converts weight into numeric values
df['Weight_kg'] = df['Weight'].str.replace('kg', '').replace('?', np.nan).astype(fl
#imputing missing weights with median
df['Weight_kg'] = df['Weight_kg'].fillna(df['Weight_kg'].median())
print("missing values in Weight_kg:", df['Weight_kg'].isnull().sum())
```

```
missing values in Weight_kg: 0
```

Median imputation was chosen instead of mean imputation beacuase several numerical variables, particulary price and storage size, exhibit right-skewed dsitribution with outliers.

In [11]:

```
df[['Weight_kg', 'Weight']].head(10)
```

Out[11]:

| | Weight_kg | Weight |
|---|---|---|
| **0** | 1.37 | 1.37kg |
| **1** | 1.34 | 1.34kg |
| **2** | 1.86 | 1.86kg |
| **3** | 1.83 | 1.83kg |
| **4** | 1.37 | 1.37kg |
| **5** | 2.10 | 2.1kg |
| **6** | 2.04 | 2.04kg |
| **7** | 1.34 | 1.34kg |
| **8** | 1.30 | 1.3kg |
| **9** | 1.60 | 1.6kg |

In [12]:
```
#extracting CPU brand and speed GHz
#CPU brand categorical
df['CPU_Brand'] = df['Cpu'].str.split().str[0]
df['CPU_GHz'] = df['Cpu'].str.extract(r'(\d\.\d)GHz').astype(float)
#imputing missing CPU speeds with median
df['CPU_GHz'] = df['CPU_GHz'].fillna(df['CPU_GHz'].median())
df[['Cpu', 'CPU_Brand', 'CPU_GHz']].head(10)
```

Out[12]:

| | Cpu | CPU_Brand | CPU_GHz |
|---|---|---|---|
| **0** | Intel Core i5 2.3GHz | Intel | 2.3 |
| **1** | Intel Core i5 1.8GHz | Intel | 1.8 |
| **2** | Intel Core i5 7200U 2.5GHz | Intel | 2.5 |
| **3** | Intel Core i7 2.7GHz | Intel | 2.7 |
| **4** | Intel Core i5 3.1GHz | Intel | 3.1 |
| **5** | AMD A9-Series 9420 3GHz | AMD | 2.5 |
| **6** | Intel Core i7 2.2GHz | Intel | 2.2 |
| **7** | Intel Core i5 1.8GHz | Intel | 1.8 |
| **8** | Intel Core i7 8550U 1.8GHz | Intel | 1.8 |
| **9** | Intel Core i5 8250U 1.6GHz | Intel | 1.6 |

CPU performance strongly influences laptop pricing.

In [13]:
```
#extracting stroage information (separating SSD and HDD into independent numeric va
df['SSD_GB'] = df['Memory'].str.extract(r'(\d+)GB (SSD|Flash)').iloc[:,0].astype(fl
df['HDD_GB'] = df['Memory'].str.extract(r'(\d+)TB HDD').astype(float) * 1024

#filling missing values with 0
df[['SSD_GB', 'HDD_GB']] = df[['SSD_GB', 'HDD_GB']].fillna(0)
df[['Memory', 'SSD_GB', 'HDD_GB']].head(10)
```

Out[13]:

| | Memory | SSD_GB | HDD_GB |
|---|---|---|---|
| **0** | 128GB SSD | 128.0 | 0.0 |
| **1** | 128GB Flash Storage | 128.0 | 0.0 |
| **2** | 256GB SSD | 256.0 | 0.0 |
| **3** | 512GB SSD | 512.0 | 0.0 |
| **4** | 256GB SSD | 256.0 | 0.0 |
| **5** | 500GB HDD | 0.0 | 0.0 |
| **6** | 256GB Flash Storage | 256.0 | 0.0 |
| **7** | 256GB Flash Storage | 256.0 | 0.0 |
| **8** | 512GB SSD | 512.0 | 0.0 |
| **9** | 256GB SSD | 256.0 | 0.0 |

Different storage types have distinct pricing effects and should not be combined.

In [14]:

```
#extracting screen resolution
df['Resolution_X'] = df['ScreenResolution'].str.extract(r'(\d+)x').astype(int)
df['Resolution_Y'] = df['ScreenResolution'].str.extract(r'x(\d+)').astype(int)

df[['ScreenResolution','Resolution_X','Resolution_Y']].head(10)
```

Out[14]:

| | ScreenResolution | Resolution_X | Resolution_Y |
|---|---|---|---|
| **0** | IPS Panel Retina Display 2560x1600 | 2560 | 1600 |
| **1** | 1440x900 | 1440 | 900 |
| **2** | Full HD 1920x1080 | 1920 | 1080 |
| **3** | IPS Panel Retina Display 2880x1800 | 2880 | 1800 |
| **4** | IPS Panel Retina Display 2560x1600 | 2560 | 1600 |
| **5** | 1366x768 | 1366 | 768 |
| **6** | IPS Panel Retina Display 2880x1800 | 2880 | 1800 |
| **7** | 1440x900 | 1440 | 900 |
| **8** | Full HD 1920x1080 | 1920 | 1080 |
| **9** | IPS Panel Full HD 1920x1080 | 1920 | 1080 |

Splits resolution into width and height. Resolution affects display quality and price, numeric separation enables regression

In [15]:

```
#convert inches to numeric
df['Inches'] = pd.to_numeric(df['Inches'], errors='coerce')
```

```
df['Inches'] = df['Inches'].fillna(df['Inches'].median())
print("missing values in Inches:", df['Inches'].isnull().sum())
```

```
missing values in Inches: 0
```

In [16]:
```
#checking to see that there are no missing rows
df.isnull().sum()
```

Out[16]:
```
Unnamed: 0          0
Company             0
TypeName            0
Inches              0
ScreenResolution    0
Cpu                 0
Ram                 0
Memory              0
Gpu                 0
OpSys               0
Weight              0
Price               0
Ram_GB              0
Weight_kg           0
CPU_Brand           0
CPU_GHz             0
SSD_GB              0
HDD_GB              0
Resolution_X        0
Resolution_Y        0
dtype: int64
```

In [17]:
```
df.describe()
```

Out[17]:

| | Unnamed: 0 | Inches | Price | Ram_GB | Weight_kg | CPU_GHz | |
|---|---|---|---|---|---|---|---|
| **count** | 1273.000000 | 1273.000000 | 1273.000000 | 1273.000000 | 1273.000000 | 1273.000000 | 12 |
| **mean** | 652.674784 | 15.131186 | 59955.814073 | 8.462687 | 2.077589 | 2.326316 | 1 |
| **std** | 376.493027 | 1.953712 | 37332.251005 | 5.564408 | 0.807491 | 0.491091 | 1 |
| **min** | 0.000000 | 10.100000 | 9270.720000 | 1.000000 | 0.000200 | 0.900000 | |
| **25%** | 327.000000 | 14.000000 | 31914.720000 | 4.000000 | 1.500000 | 2.200000 | |
| **50%** | 652.000000 | 15.600000 | 52161.120000 | 8.000000 | 2.040000 | 2.500000 | 1 |
| **75%** | 980.000000 | 15.600000 | 79333.387200 | 8.000000 | 2.320000 | 2.700000 | 2 |
| **max** | 1302.000000 | 35.600000 | 324954.720000 | 64.000000 | 11.100000 | 3.600000 | 5 |

In [18]:
```
from scipy.stats import skew, kurtosis

target_col = 'Price'

#calculate central tendancy
```

```python
price_mean = df[target_col].mean()
price_median = df[target_col].median()
price_std = df[target_col].std()
price_var = df[target_col].var()
price_min = df[target_col].min()
price_max = df[target_col].max()
price_range = price_max - price_min
price_skewness = skew(df[target_col].dropna())
price_kurtosis = kurtosis(df[target_col].dropna())

print(f"\n Measure of Central Tendency:")
print(f" Mean: {price_mean:.2f}")
print(f" Median: {price_median:.2f}")
print(f" Mode: Multiple values (continuous data)")

print(f"\n Measures of Spread:")
print(f"  Standard Deviation: {price_std:.2f}")
print(f"  Variance:           {price_var:.2f}")
print(f"  Range:              {price_range:.2f}")
print(f"  Min:                {price_min:.2f}")
print(f"  Max:                {price_max:.2f}")

#interquartile range
q75, q25 = df[target_col].quantile(0.75), df[target_col].quantile(0.25)
print(f"  IQR:                {q75 - q25:.2f}")
```

```
Measure of Central Tendency:
Mean: 59955.81
Median: 52161.12
Mode: Multiple values (continuous data)

Measures of Spread:
 Standard Deviation: 37332.25
 Variance:           1393696965.07
 Range:              315684.00
 Min:                9270.72
 Max:                324954.72
 IQR:                47418.67
```

The mean laptop price (rupees) is ₹59,956 while the median is ₹52,161. The mean is significantly higher than the median by (₹7,795 or 15%) is a clear indicator of a right-skewed distribution. This means that while most laptops cluster in the budget-to-midrange segment, expensive high-end models pull the average upward.

This pattern is typically in consumer electronics marlers where: -Mass-market producrs dominate sales volume(pulling median down) -Premium products have dipropotinate pricing (pulling mean up) -The market serves both budget-conscious and performance-focused segments

The standard deviation of ₹37,332 represents 62% of the mean, indicating substantial price variability. This high variability is expected: -Reflects diverse product segment (budget notebooks to gaming workstations) -Suggests different specifications create distinct price tiers

The range spans from ₹9,271(entry-level budget laptops) to ₹324,955 (premium workstations), covering a 35x price difference. The IQR of ₹47,419 shows that the middle 50% of laptops fall within roughly 1.5x price range, while outliers in both directions extend the total range.

In [19]:
```
print(f"Skewness: {skew(df[target_col])}")
print(f"Kurtosis: {kurtosis(df[target_col])}")
```

```
Skewness: 1.5279779642160025
Skewness: 4.394072588145356
```

Skewness = 1.53 is positive, which confirms significant right-skewness:

- Values > 0.5 indicate moderate to high positive skewness
- Right tail (epenive laptops) is longer than the left tail
- More laptop prices below the mean than above it
- Distribution is asymmetric, with the bulk of data on the left

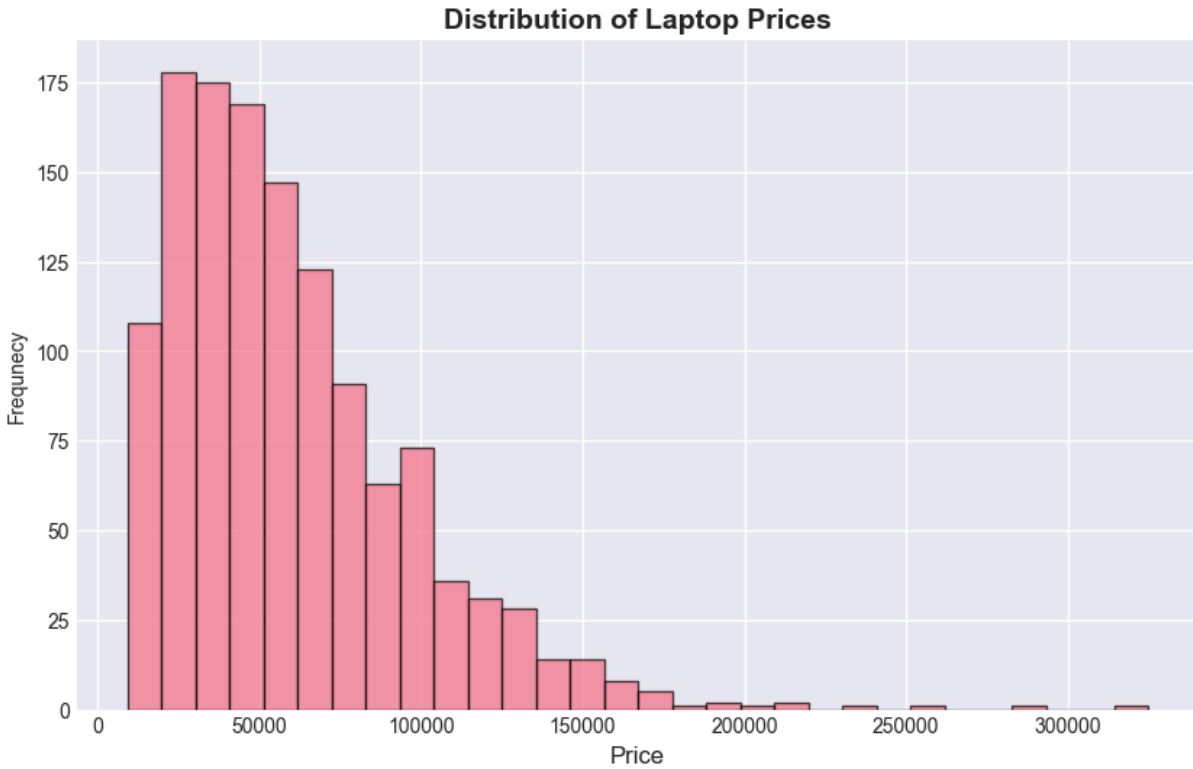
Kurtosis = 4.39 (positive):

- Values > 3 indicate heavier tails than the normal distribution
- More extreme values (very cheap and very expensive) than expected
- Market competition clusters products in popular price points

## Data Visualisation

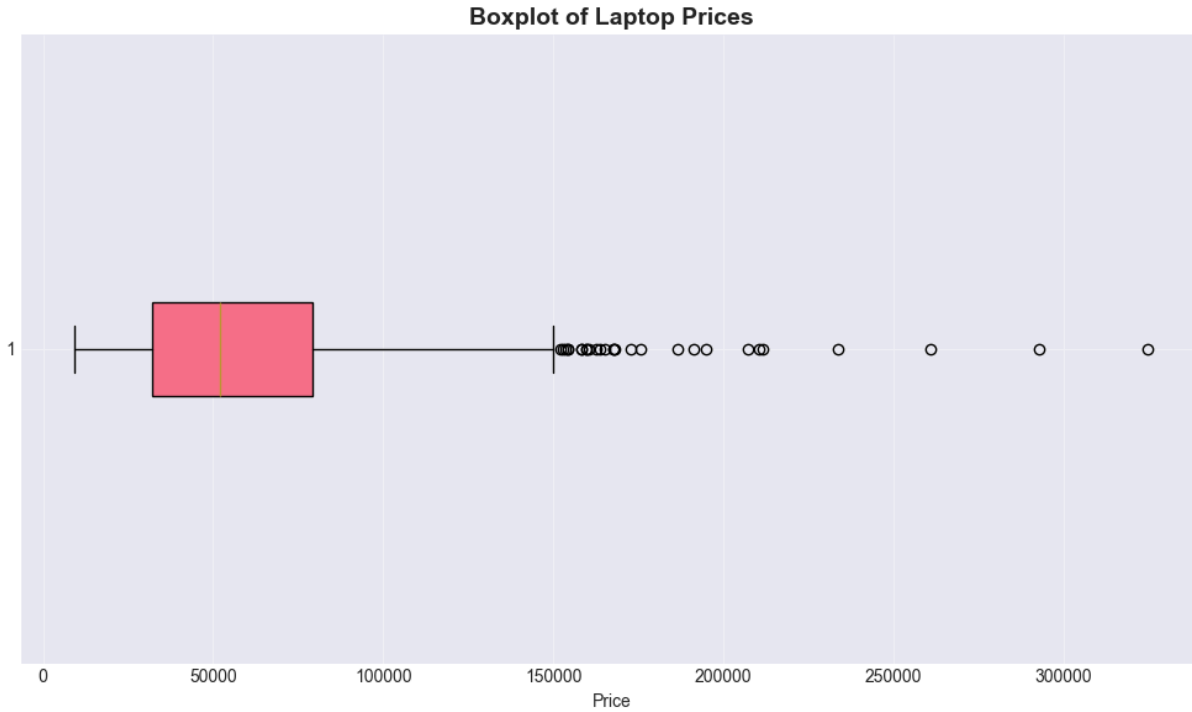
The histogram shows that the energy consumption

In [20]:
```
plt.figure(figsize=(10,6))
plt.hist(df["Price"], bins=30, edgecolor='black', alpha=0.7)
plt.title("Distribution of Laptop Prices", fontsize=14, fontweight='bold')
plt.xlabel("Price", fontsize=12)
plt.ylabel("Frequnecy")
plt.show()
```



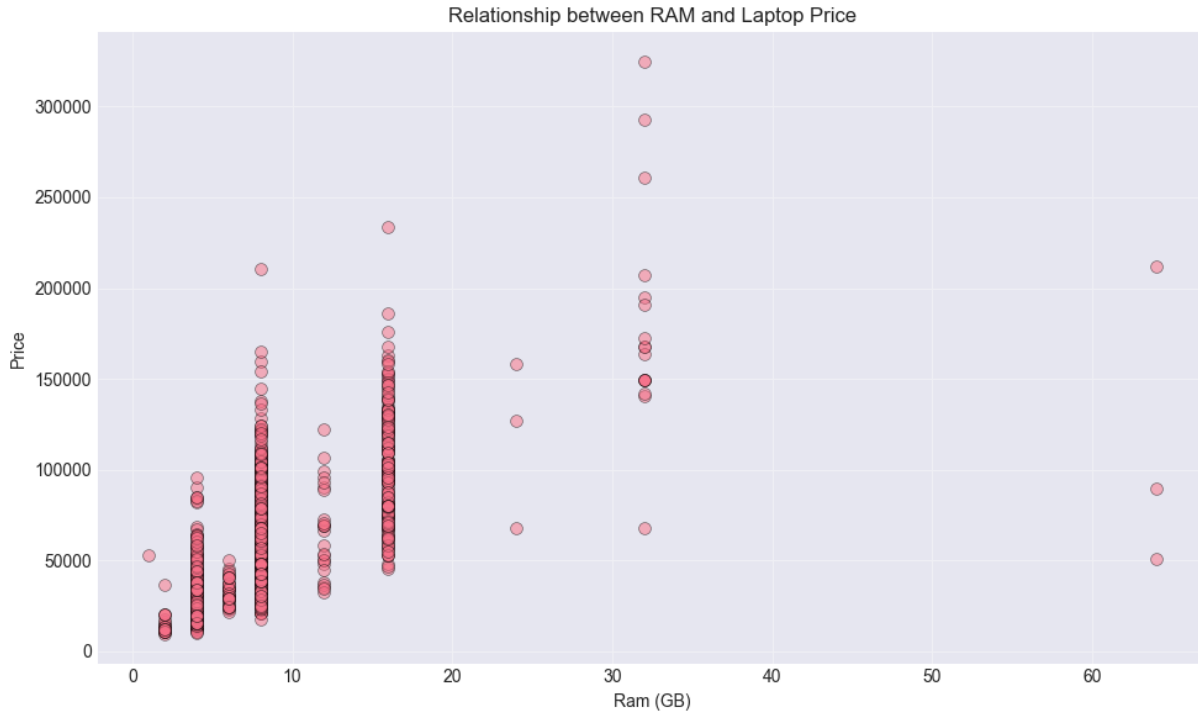

The histogram shows that the laptop prices follow a right-skewed distribution with most laptop prices between 30,000-80,000. The postive skewness confirms that there are more budget laptop with a long tail of expesve high-end models. The distribution has a clear peak around 40,000-50,000 prepresenting the most common price point.

In [21]:
```
plt.figure(figsize=(10,6))
plt.boxplot(df['Price'], vert=False, patch_artist=True)
plt.title("Boxplot of Laptop Prices", fontsize=14, fontweight='bold')
plt.xlabel("Price")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



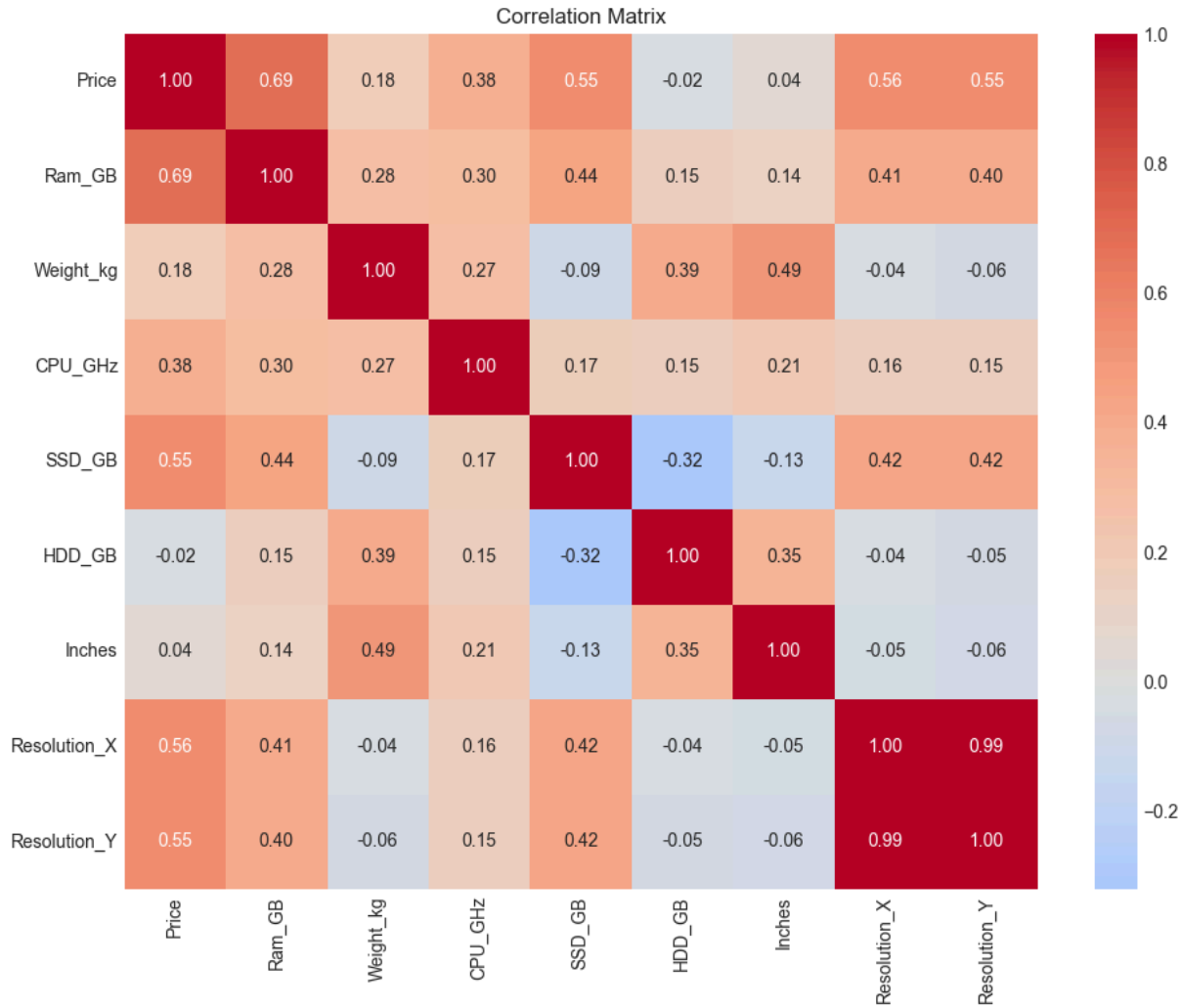

The boxplot reveals significant outliers above 200,000, representing premium gaming and workstation laptops. The median price (~52,000) is lower than the mean (~60,000) due to right skewness, confriming the presence of expenive outliers pulling the average up.

In [22]:
```
plt.figure(figsize=(10,6))
plt.scatter(df['Ram_GB'], df['Price'], alpha=0.5, s=50, edgecolors='black', linewid
plt.xlabel('Ram (GB)')
plt.ylabel('Price')
plt.title('Relationship between RAM and Laptop Price')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```




This scatter plot shows a clear positive relationship between RAM and price. Laptops with 16GB+ RAM command significantly higher prices. The correlation of price indicates RAM is an important pricing factor. However, we see some overlap, suggesting other factors also influence price.

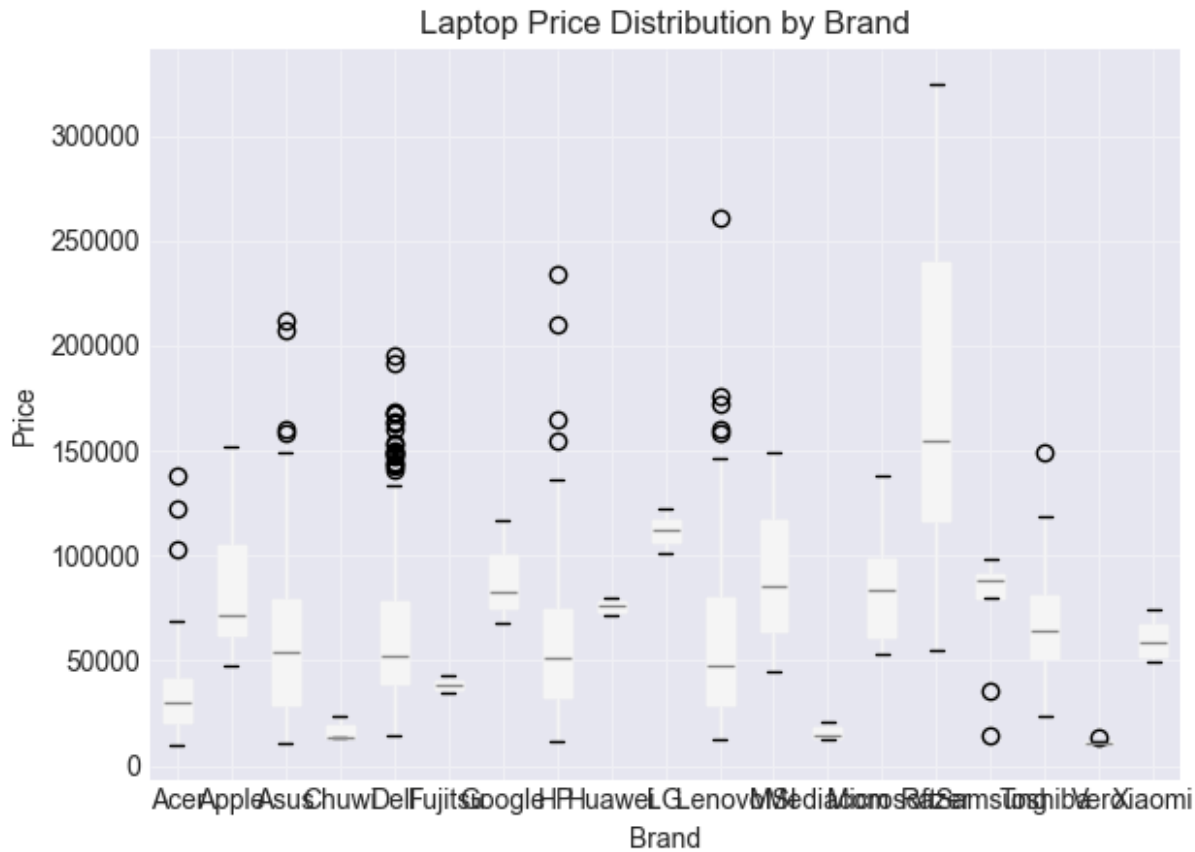
In [23]:
```
numerical_features = ['Price','Ram_GB','Weight_kg', 'CPU_GHz', 'SSD_GB', 'HDD_GB',
corr = df[numerical_features].corr()
plt.figure(figsize=(10,8))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0)
plt.title('Correlation Matrix')
plt.tight_layout()
plt.show()
```




The correlation matrix reveals that CPU_GHz shows the strongest correlation with Price, followed by RAM_GB and SSD_GB. Resolution_X and Resolution_Y are highly correlated with each other. This helps us understand which features are most predictive.

In [24]:
```
plt.figure(figsize=(10,8))
df.boxplot(column="Price", by="Company", patch_artist=True)
plt.xticks(rotation=45)
plt.title("Laptop Price Distribution by Brand")
plt.suptitle('')
plt.xlabel("Brand")
plt.ylabel("Price")
plt.xticks(rotation=0)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
<Figure size 1000x800 with 0 Axes>
```




This boxplot shows significant price variation across brands. Apple laptops command premium prices (median>100,000), while other brands like Acer offer more budget-friendly options. This suggests brand is an important categorical feature that should be included in the model.

## Machine learning model

In [25]:
```
#creating features
features = ['Ram_GB', 'Weight_kg', 'CPU_GHz', 'SSD_GB', 'Inches', 'Resolution_X', '
X = df[features]
y = df['Price']
```

This step defines the input features (X) and the target variable (y) for the model. The selected features include hardware specifications (RAM, CPU speed, storage, screen size and resolution) and categorical attributes (brand, laptop type, CPU brand). These variables were chosen based on domain knowledge and earlier correlation analysis, which showed that they have meaningful relationship with laptop pricing. The target variable is Price, a continuous numeric variable, making this a supervised regression model.

In [26]:
```
X = pd.get_dummies(X, drop_first=True)
```

Categorical variables cannot be used by linear regression model. One-hot encoding converts each cetegory into binary indicator variables.

- drop_first=True is used to aviod dummy variable trap, which prevents multicollinearity by removing one refernce category per categorical feature.
- This ensures numerical stability and improves model interpretability.

In [27]:
```
X.dtypes.head()
```

Out[27]:
```
Ram_GB         int32
Weight_kg    float64
CPU_GHz      float64
SSD_GB       float64
Inches       float64
dtype: object
```

This step confirms that all features are now in a numeric format, which is required for linear regression. Integer and floating-point data types indicate that the dataset is ready, as done at the start to ensure it is ready for modelling.

In [28]:
```
X = X.fillna(X.median())
X.isnull().sum()
```

Out[28]:
```
Ram_GB                 0
Weight_kg              0
CPU_GHz                0
SSD_GB                 0
Inches                 0
Resolution_X           0
Resolution_Y           0
Company_Apple          0
Company_Asus           0
Company_Chuwi          0
Company_Dell           0
Company_Fujitsu        0
Company_Google         0
Company_HP             0
Company_Huawei         0
Company_LG             0
Company_Lenovo         0
Company_MSI            0
Company_Mediacom       0
Company_Microsoft      0
Company_Razer          0
Company_Samsung        0
Company_Toshiba        0
Company_Vero           0
Company_Xiaomi         0
TypeName_Gaming        0
TypeName_Netbook       0
TypeName_Notebook      0
TypeName_Ultrabook     0
TypeName_Workstation   0
CPU_Brand_Intel        0
CPU_Brand_Samsung      0
dtype: int64
```

Final check to confirm that no missing values remain, ensuring compability with scikit-learn models.

In [29]:
```
#defining features x and target y
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

In [30]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

X_train.shape, X_test.shape
```

Out[30]:
```
((1018, 32), (255, 32))
```

The dataset is split into training(80%) and testing (20%) subsets.

- The training set is used to fit the model
- The testing set is used to evaluate performance on unseen data
- random_state=42 ensures reproducibility, meaning results can be consistenlty replicated

The shape confirms the there is 1018 samples and 255 testing samples. There are 32 total features after one-hot encoding.

# Linear regression model

In [31]:
```
#building linear regression model
model = LinearRegression()
model.fit(X_train, y_train)

#make predictions on validation set
y_pred = model.predict(X_test)

#evaluating model performance
rmse = mean_squared_error(y_test, y_pred, squared=False)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"R^2 Score: {r2:.4f}")
```

```
Root Mean Squared Error (RMSE): 20833.82
Mean Absolute Error (MAE): 13996.74
R^2 Score: 0.7008
```

Trains a linear regression model and evalutes it using RMSE, MAE and R^2

In [32]:
```
num_cols = ['Ram_GB', 'Weight_kg', 'CPU_GHz', 'SSD_GB', 'Inches', 'Resolution_X', '

scaler = StandardScaler()
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])
```

Feature scaling was applied to the numerical variables using Standard Scaler to normalise feature magnitudes. This ensures that the variables measured on different scaled (eg. SSD size vs screen resolution) does not disproportionately influence coeffcient interpretation.

In [33]:
```
#modelling coefficients
coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_
}).sort_values('Coefficient', key=abs, ascending=False)

coef_df
```

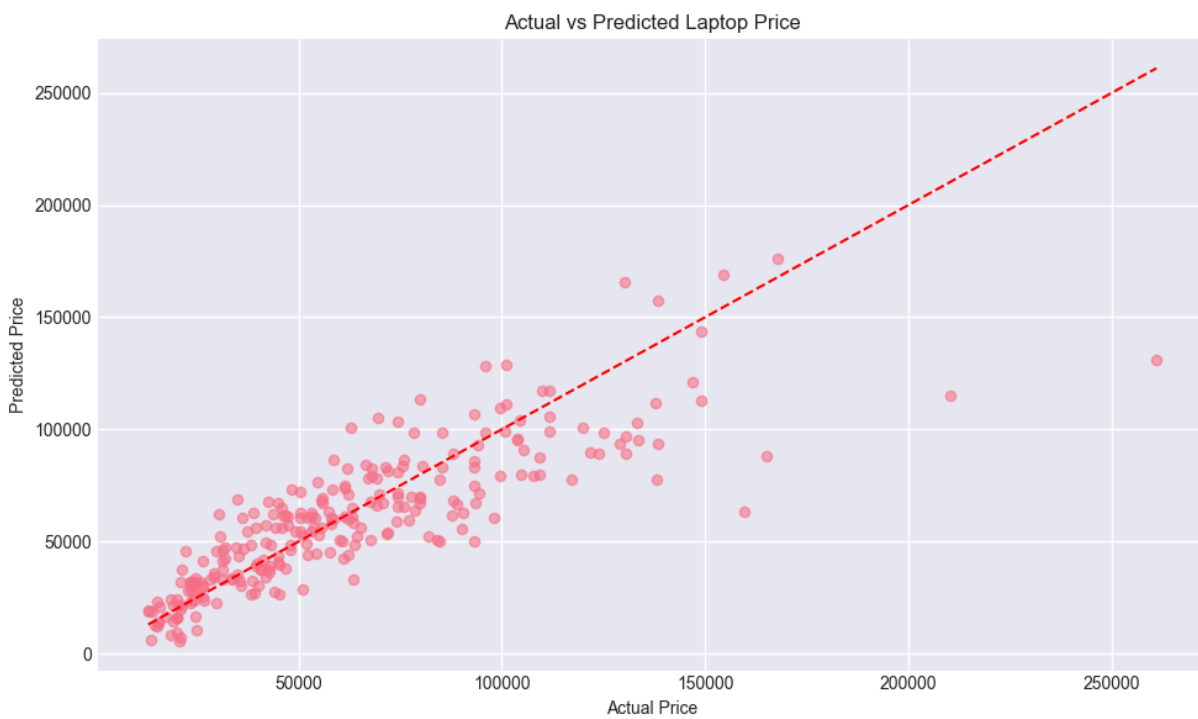
Out[33]:

| | Feature | Coefficient |
|---|---|---|
| **20** | Company_Razer | 72465.393033 |
| **29** | TypeName_Workstation | 35970.253781 |
| **15** | Company_LG | 34703.658042 |
| **12** | Company_Google | 32083.936110 |
| **19** | Company_Microsoft | 19352.744537 |
| **7** | Company_Apple | 18869.338181 |
| **22** | Company_Toshiba | 18175.191239 |
| **21** | Company_Samsung | 16703.193762 |
| **30** | CPU_Brand_Intel | 14255.714623 |
| **27** | TypeName_Notebook | -13176.290069 |
| **9** | Company_Chuwi | -12500.780335 |
| **13** | Company_HP | 11397.152025 |
| **17** | Company_MSI | 10859.777846 |
| **2** | CPU_GHz | 9285.640852 |
| **23** | Company_Vero | -8790.388147 |
| **31** | CPU_Brand_Samsung | -7817.956021 |
| **14** | Company_Huawei | 7352.162974 |
| **18** | Company_Mediacom | -7090.813996 |
| **10** | Company_Dell | 6828.193353 |
| **16** | Company_Lenovo | 5424.150478 |
| **8** | Company_Asus | 4957.319143 |
| **26** | TypeName_Netbook | -4925.228717 |
| **24** | Company_Xiaomi | 4890.606085 |
| **1** | Weight_kg | 3734.597143 |
| **25** | TypeName_Gaming | 2452.793239 |
| **0** | Ram_GB | 2137.053822 |
| **11** | Company_Fujitsu | 1695.781115 |
| **28** | TypeName_Ultrabook | 1581.409924 |
| **4** | Inches | 88.941413 |
| **3** | SSD_GB | 45.512236 |

| | Feature | Coefficient |
|---|---|---|
| **5** | Resolution_X | 21.173151 |
| **6** | Resolution_Y | -10.675002 |

The coefficient values represent the expected change in laptop price(in rupees) for a one-unit increase in the corresponding feature, holding all other variables constant. For categorical variables, coefficients represent the price difference relative to the reference category removed during one-hot encoding. Large positive coefficients for brand such as Razer and Apple reflect strong brand premiums, while negative coefficients indicate lower-priced segments.

In [39]:
```
#actual vs predicted
plt.figure(figsize=(10,6))
plt.scatter(y_test, y_pred, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.xlabel('Actual Price')
plt.ylabel('Predicted Price')
plt.title('Actual vs Predicted Laptop Price')
plt.tight_layout()
plt.show()
```




Analysis: The scatter plot shows predictions plotted against actual price, with the red dashed line representing predictions (predicted = actual). Observation: -Most points cluster near the red line: Indicating good prediction accuracy for the majority of laptops

The visual confirms the numerical metrics (R^2 and RMSE). Clustering around the diagonal shows the model successfully learned the pricing relationship. Any systematic deviations would be visible as curved patterns or zones with consistent over/under-prediction.

## Model Evaluation

In [35]:
```
cv_rmse = np.sqrt(-cross_val_score(model, X, y, cv=5, scoring='neg_mean_squared_err

print("CV RMSE:", cv_rmse.mean())
```

```
CV RMSE: 20560.636011538238
```

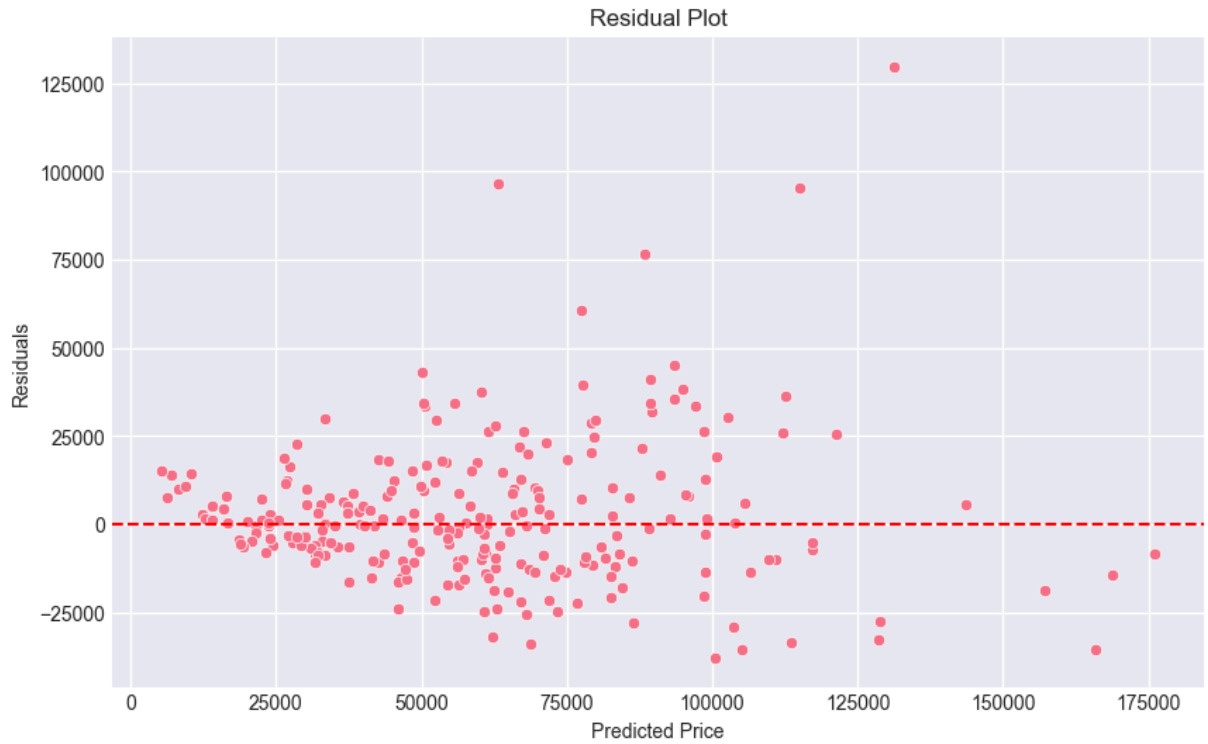
This code perfroms 5-fold-cross-validation on the linear regression model using Root Mean Squared Error (RMSE) as the evaluation metric.

- The dataset is split into five folds
- The model is trained on four folds and tested on the remaining fold
- This process is repeated five times so that each fold is used for validation once
- The average RMSE across all folds is then calculated

RMSE is used because:

- Same units as the target, as it is expressed in rupees, and is interpretable
- Penalises large errors as it is critical for avoiding major price mispredictions
- Industry standards that are commonly used in pricing models for comparison

In [36]:
```
#residual plot
residuals = y_test - y_pred
plt.figure(figsize=(10,6))
sns.scatterplot(x=y_pred, y=residuals)
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel('Predicted Price')
plt.ylabel('Residuals')
plt.title('Residual Plot')
plt.show()
```




The residual plot visualises the difference actual and predicted prices against the predicted values.

- Residual are randomly scattered acround zero, with no clear pattern or curvature
- There is no strong funnel shape, suggesting approximately constant variance of errors
- The model does not consistently over-predict or under-predict prices

The residual plot confirms that the model does not consistently over-predict prices. Errors appear randomly rather than systematically. Linear regression is an appropriate modelling choice for this problem. Any remaining prediction errors are likely due to unmodelled factors, such as GPU performance, build quality, or brand-specific strategies, rather than a flaw in the regression approach itself.

In [37]:
```
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)

X_train_poly, X_test_poly, y_train_poly, y_test_poly = train_test_split(X_poly, y,

model_poly = LinearRegression()
model_poly.fit(X_train_poly, y_train_poly)
y_pred_poly = model_poly.predict(X_test_poly)

rmse_poly = mean_squared_error(y_test_poly, y_pred_poly, squared=False)
r2_poly = r2_score(y_test_poly, y_pred_poly)

print(f"Polynomial RMSE: {rmse_poly:.2f}")
print(f"Polynomial R^2: {r2_poly:.4f}")
print(f"\nImprovemnts: {((rmse - rmse_poly)/rmse * 100):.2f}%")
```

```
Polynomial RMSE: 400699.38
Polynomial R^2): -109.6792

Improvemnts: -1823.31%
```

## Model Evaluation:

The linear regression model demonstrated strong predictive performance, with RMSE and R^2 values indicating a good fit to the data. The use of RMSE is appropriate because it penalises large pricing errors, which are particularly costly in retail pricing scenarios. Cross-validation confirmed that the model generalises well across different data splits.

Polynomial feature expansion significantly worsened model performance, as evidenced by a dramatic increase in RMSE and a negative R^2 value. This indicates severe overfitting due to high feature dimensionality after polynomial expansion. As a result, the linear regression model is preferred for this dataset due to its superior generalisation and interpretability.

While the model performs well, it does not include certain influential factors such as GPU performance, build quality, battery life, release year, or market demand dynamics. Additionally, linear regression assumes linear relationships and may underperform for premium or niche laptops with non-linear pricing stategies. Future work could explore regularised models(Ridge/Lasso), tree-based methods, or feature engineering for GPU and brand sub-categories to future improve predictive accuracy.

### References:

https://www.kaggle.com/datasets/ehtishamsadiq/uncleaned-laptop-price-dataset

GeeksforGeeks (2024). Calculating RMSE Using Scikitlearn. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/machine-learning/calculating-rmse-using-scikit-learn/.

Sharma, P. (2022). Different Types of Regression Models. [online] Analytics Vidhya. Available at: https://www.analyticsvidhya.com/blog/2022/01/different-types-of-regression-models/.